

Spring Security + JWT Authentication Flow

This document explains the complete authentication and authorization flow using Spring Security and JWT in a stateless REST application. It is written for beginners who have recently learned Spring Security.

1. What Problem Are We Solving?

We want to secure REST APIs without using server-side sessions. Users should log in once using username and password, receive a JWT token, and then use that token to access protected APIs.

2. Stateless Authentication (Core Idea)

In a stateless system, the server does not remember who the user is. Every request must contain proof of authentication. JWT (JSON Web Token) acts as that proof.

- No HTTP session is created
- No server-side login memory
- Every protected request must send JWT

3. Main Components

- SecurityConfig – defines security rules and filters
- AuthController – handles register and login endpoints
- AuthenticationManager – validates username and password
- UserDetailsService – loads user data from database
- JwtService – generates and validates JWT tokens
- JwtFilter – authenticates every request using JWT

4. Login Flow (Username & Password)

The login flow happens only once. It uses Spring Security's AuthenticationManager.

```
Client
|
|  POST /auth/login (username, password)
v
AuthController
|
|  authenticationManager.authenticate()
v
AuthenticationManager
|
v
DaoAuthenticationProvider
|
v
UserDetailsService (DB lookup)
|
```

```

      v
PasswordEncoder (BCrypt check)
      |
      v
Authentication SUCCESS
      |
      v
JwtService.generateToken()
      |
      v
JWT returned to Client

```

5. JWT Token

JWT is a digitally signed token that contains user identity information. It is not stored on the server.

- Subject (email / username)
- Issued time (iat)
- Expiration time (exp)
- Signature (prevents tampering)

6. Protected API Flow (JWT Authentication)

Every protected request must send the JWT in the Authorization header.

```

Client
  |
  | GET /api/hello
  | Authorization: Bearer <JWT>
  v
Security Filter Chain
  |
  v
JwtFilter
  |
  |-- Extract JWT
  |-- Validate signature & expiry
  |-- Load user from DB
  |-- Set SecurityContextHolder
  |
  v
Authorization Check
  |
  v
Controller

```

7. What Happens Without JWT?

If a protected endpoint is accessed without JWT, authentication fails and the controller is never executed.

8. Role of SecurityContextHolder

SecurityContextHolder stores the Authentication object for the current request. If it is not set, Spring Security treats the user as anonymous.

9. One-Page Mental Model

REGISTER:

Client -> Controller -> Save User

LOGIN:

Client -> Controller
-> AuthenticationManager
-> UserDetailsService
-> PasswordEncoder
-> JWT returned

EVERY PROTECTED REQUEST:

Client -> JwtFilter
-> Validate JWT
-> SecurityContext set
-> Authorization
-> Controller

10. Final Takeaway

Authentication happens only when Spring Security is explicitly told who the user is. JWT + JwtFilter provide that information for every request in a stateless system.